

Programmierung 2 - Sommersemester 2026

Prof. Dr. Markus Esch

Übung Nr. 7 Abgabe KW 25

1. Aufgabe

- (a) Erstellen Sie ein Programm zur Verwaltung des Inventars eines Einzelhändlers, verwenden Sie dabei Java Collections. Das System sollte Informationen zu Produkten speichern und grundlegende Operationen wie Hinzufügen, Entfernen, Suchen und Sortieren von Produkten ermöglichen. Das Programm sollte zudem über ein Command Line Interface (CLI) verfügen. Setzen Sie dabei folgende Anforderungen um.

- i. Erstellen Sie eine Klasse `Product` mit den folgenden Eigenschaften:

- `int productId`
- `String name`
- `String category` (Kategorie)
- `double price` (Preis)
- `int quantity` (Menge)

- ii. Erstellen Sie eine Klasse `Inventory`, die eine Collection von Produkten verwaltet. Sie müssen die geeignete Collection selbst auswählen und begründen warum Sie diese gewählt haben. Die Collection sollte folgende Anforderungen erfüllen:

- Schnelles Hinzufügen und Entfernen von Produkten
- Effizientes Suchen nach Produkten anhand der Produkt-ID

Die `Inventory`-Klasse sollte folgende Methoden enthalten:

- `void addProduct(Product product)`: Fügt ein Produkt zum Inventar hinzu.
- `boolean removeProduct(int productId)`: Entfernt ein Produkt anhand der `productId`. Gibt `true` zurück, wenn das Produkt erfolgreich entfernt wurde, ansonsten `false`.
- `Product findProductById(int productId)`: Sucht ein Produkt anhand der `productId`. Gibt das Produkt zurück, wenn es gefunden wurde, ansonsten `null`.
- `List<Product> findProductsByCategory(String category)`: Gibt eine Liste von Produkten zurück, die einer bestimmten Kategorie angehören.
- `List<Product> getAllProducts()`: Gibt eine Liste aller Produkte zurück.
- `List<Product> sortProductsByName()`: Gibt eine Liste der Produkte im Inventar sortiert nach Namen zurück.
- `List<Product> sortProductsByPrice()`: Gibt eine Liste der Produkte im Inventar sortiert nach Preis zurück.
- `List<Product> getLowStockProducts(int threshold)`: Gibt eine Liste von Produkten zurück, deren Menge unter einem bestimmten Schwellenwert liegt.

- `List<Product> filterProducts(Predicate<Product> predicate)`: Gibt eine Liste von Produkten zurück, die das übergebene Prädikat erfüllen.
 - `void applyToProducts(Consumer<Product> consumer)`: Wendet eine Operation auf alle Produkte im Inventar an.
- iii. Erstellen Sie ein Command Line Interface (CLI), das die folgenden Optionen bietet:
1. Produkt hinzufügen
 2. Produkt entfernen
 3. Produkt nach ID suchen
 4. Produkte nach Kategorie anzeigen
 5. Alle Produkte anzeigen
 6. Produkte nach Namen sortieren
 7. Produkte nach Preis sortieren
 8. Produkte mit niedrigen Lagerbeständen anzeigen
 9. Produkte nach benutzerdefiniertem Filter anzeigen
 10. Preise aller Produkte um einen Prozentsatz erhöhen
 11. Programm beenden
- iv. Implementieren Sie das Hauptprogramm, das das CLI verwendet, um mit dem `Inventory` zu interagieren. Das Hauptprogramm sollte in der Lage sein, Eingaben vom Benutzer entgegenzunehmen und die entsprechenden Methoden des `Inventory` aufzurufen.
- (b) Erweitern Sie ihr Programm so, dass es möglich ist, in der Einfügereihenfolge über die Produkte im Inventar zu iterieren.

2. Aufgabe

Implementieren Sie ein Programm mit einer `Producer`- und einer `Consumer`-Klasse. Der `Producer` erzeugt zufällige Integer, welche in einer geeigneten `Collection` gespeichert werden. Der `Consumer` entnimmt die Integer aus der `Collection` und berechnet die Quersumme. In einer Schleife sollen `Producer` und `Consumer` in einer zufälligen Reihenfolge aufgerufen werden. Etwa wie folgt:

```
Random ran = new Random();
for(int i = 0; i<10000; i++)
{
    if(ran.nextInt(2) > 0 )
        // Erzeugen eines neuen Integers durch den Producer und speichern
        // in einer Collection
    else
        // Entnehmen eines Integeres aus der Collection und Berechnung der
        // Quersumme durch den Consumer
}
```

Beachten Sie bei der Implementierung folgende Aspekte:

- (a) Die Entnahme der Integer aus der `Collection` soll entweder nach FIFO-Ordnung oder entsprechend der natürlichen Sortierung der Integer erfolgen. Welche der beiden Varianten genutzt wird, soll beim Programmstart als Parameter übergeben werden. Überlegen Sie, welche `Collection` für die jeweilige Variante am besten geeignet ist.

- (b) Die Producer-Klasse besitzt eine Methode `produce` welche einen zufälligen Integer zwischen 0 und 1000 erzeugt und zurückgibt.
- (c) Die Consumer-Klasse besitzt folgende Methoden:
- `consume`: Diese Methode nimmt einen Integer entgegen und berechnet die Quersumme. Für jede Berechnung soll außerdem der Zeitstempel der Berechnung gespeichert werden. Nutzen Sie die Methode `System.currentTimeMillis()`, um den Zeitstempel zu erhalten.
Die Zeitstempel müssen so gespeichert werden, dass für eine gegebene Quersumme alle zugehörigen Zeitstempel effizient zugegriffen werden können. Zugehörige Zeitstempel heißt: Die Zeitstempel aller Berechnungen, die zu der gegebenen Quersumme geführt haben. Es soll außerdem möglich sein effizient zu zählen, wie häufig eine gegebene Quersumme berechnet wurde. Überlegen Sie welche Datenstruktur dies am besten unterstützt. Hinweis: Sie müssen eine Map und eine Collection in geeigneter Weise miteinander kombinieren.
 - `numberOfDifferentResults`: gibt an, wie viele unterschiedliche Quersummen berechnet wurden
 - `numberOfOccurrences`: gibt für einen gegebenen Integer an, wie häufig dieser als Ergebnis einer Berechnung vorkam
 - `getCrossTotalsAscending`: gibt eine Collection zurück, welche die berechneten Quersummen in aufsteigender Reihenfolge enthält
 - `getCrossTotalsDescending`: gibt eine Collection zurück, welche die berechneten Quersummen in absteigender Reihenfolge enthält.
 - `getTimestampsForResult`: nimmt einen Integer entgegen und gibt eine Collection zurück, welche alle zugehörigen Zeitstempel enthält. D.h. die Zeitstempel der Berechnungen, die zu dem gegebenen Ergebnis geführt haben.

Implementieren Sie eine Testklasse, um Ihre Implementierung zu testen.

Hinweis: Ein wichtiger Aspekt der Bewertung ihrer Lösung ist, ob Sie geeignete Datenstrukturen gewählt und diese in geeigneter Weise genutzt haben. Soweit es möglich ist, sollen Sie die Funktionalität der Java-Collections nutzen, um die Anforderungen umzusetzen; z.B., um die Zeitstempel in aufsteigender Reihenfolge zu erhalten.

Lernziele:

Nach der Bearbeitung des Übungsblattes sollten Sie ...

- ... in der Lage sein für ein gegebenes Problem eine geeignete Collection auszuwählen.
- ... Collections mit dem für den jeweiligen Anwendungsfall benötigten Typ-Parameter instanziiieren können.
- ... die grundlegenden Methoden der Collection-Klassen kennen und nutzen können.
- ... Collections in einer Anwendung nutzen können, um Objekte zu verwalten.
- ... den Vorteil von Collections gegenüber Arrays benennen können.

Zusatzaufgaben

Die folgenden Aufgaben sind Zusatzaufgaben und nicht Teil des Peer Reviews! Sie können die Aufgaben freiwillig bearbeiten, um die Inhalte weiter zu üben. Selbstverständlich können Sie auch Unterstützung und Beratung zu den Zusatzaufgaben erhalten.

1. Aufgabe

- (a) Erstellen Sie ein Programm zur Verwaltung eines Kundenservice-Management-Systems. Das System sollte in der Lage sein, Kundenanfragen in einer Warteschlange zu verwalten, wobei die Anfragen in der Reihenfolge ihrer Eingänge bearbeitet werden. Folgende Anforderungen sollen umgesetzt werden:
 - i. Erstellen Sie eine Klasse `CustomerRequest` mit den folgenden Eigenschaften:
 - `int requestId`
 - `String customerName` (Name des Kunden)
 - `String requestDetail` (Details der Anfrage)
 - `String priority` (Priorität der Anfrage, z.B. "hoch", "mittel", "niedrig")
 - ii. Erstellen Sie eine Klasse `RequestQueue`, die eine Warteschlange von Anfragen verwaltet. Wählen Sie die geeignete Queue-Implementierung selbst aus und begründen Sie Ihre Wahl. Die Warteschlange sollte folgende Anforderungen erfüllen:
 - Effiziente Bearbeitung der Anfragen in der Reihenfolge ihres Eingangs (FIFO-Prinzip).
 Die `RequestQueue`-Klasse sollte folgende Methoden enthalten:
 - `void addRequest(CustomerRequest request)`: Fügt eine Anfrage zur Warteschlange hinzu.
 - `CustomerRequest processNextRequest()`: Entfernt und gibt die nächste Anfrage in der Warteschlange zurück.
 - `CustomerRequest peekNextRequest()`: Gibt die nächste Anfrage in der Warteschlange zurück, ohne sie zu entfernen.
 - `int getQueueSize()`: Gibt die Anzahl der Anfragen in der Warteschlange zurück.
 - `List<CustomerRequest> getAllRequests()`: Gibt eine Liste aller Anfragen in der Warteschlange zurück.
 - iii. Testen Sie ihre Warteschlange mithilfe eines CLI. Hierzu können Sie das in Moodle bereitgestellte Template `CustomerService.java` nutzen.
 - iv. Implementieren Sie JUnit-Tests für die `RequestQueue`.
- (b) Ändern Sie ihre Implementierung so, dass die Requests nicht in FIFO-Ordnung, sondern anhand der Priorität bearbeitet werden.